

OVAL-NG

A pragmatic modernization of the Open Vulnerability and Assessment Language — easier to author, implement and maintain

A JSON-native, modular re-framing of OVAL that keeps its proven evaluation semantics while shedding two decades of XML-schema weight.

Scope. This document analyses the OVAL language as of the 2024–2025 revival (OVAL 5.12 and the OVAL 6.0 major release), reviews how OVAL is implemented and consumed in practice — including first-hand experience from the XORCISM platform and its endpoint agent — surveys the current open-source scanner landscape, and proposes a concrete, backward-bridgeable next-generation design. It is a discussion draft intended to inform the OVAL Community and SCAP 3.0 efforts.

Contents

1	Executive summary	3
2	Background: OVAL and its 2024–2025 revival	3
3	OVAL in practice — the XORCISM experience	4
4	The open-source scanner landscape	5
5	Problem analysis — ten friction points	6
6	Design goals	7
7	Proposal: OVAL-NG	7
8	Worked examples	10
9	Why this is easier to implement & maintain	12
10	Migration & adoption path	12
11	Governance & versioning	13
12	Risks & open questions	13
	Appendix A OVAL 5.x → OVAL-NG construct map	14

1 Executive summary

OVAL remains the most precise open language for *declaratively* describing how to test a machine state for a vulnerability, a misconfiguration, a patch, or an inventory fact. Its evaluation semantics — a typed object/state model with an explicit result algebra — are genuinely good and worth preserving. What holds OVAL back is not its logic but its **encoding and its surface area**: a sprawling family of platform-specific XML Schemas that are hard to author by hand, expensive to implement, and slow to evolve.

The 2024–2025 revival (OVAL 5.12, then the OVAL 6.0 major release with "encapsulated definitions") shows healthy community energy, but it doubles down on the XML model and breaks 5.x compatibility — while the dominant open-source evaluator, OpenSCAP, still ships OVAL 5.11.3. The result is a widening gap between the *specification* and the *tooling that real programmes depend on*.

This proposal — "OVAL-NG" — keeps OVAL's evaluation model and re-expresses it as a small, JSON-native, modular standard. The core is a handful of JSON Schema-validated object kinds; platform coverage moves into versioned *probe packs* registered by capability rather than hard-coded into one monolithic schema; criteria become a readable expression tree; and security-data linkage (CVE, CPE 2.3, CVSS, EPSS, CWE) is first-class. A deterministic, lossless bridge translates existing OVAL 5.x/6.0 XML into OVAL-NG so no content is stranded. The design target is blunt: **a competent engineer should be able to write a conformant Core evaluator in a few hundred lines, and a security analyst should be able to author a check without a schema IDE.**

2 Background: OVAL and its 2024–2025 revival

OVAL was created at MITRE (2002) to standardise three things: how to **represent** system configuration information for testing, how to **express** a specific machine state, and how to **report** the result of an assessment. A definition references *tests*; each test pairs an *object* (what to collect) with a *state* (what to compare against); definitions combine tests through a *criteria* tree using a typed result algebra. The same model underpins SCAP alongside XCCDF, CPE, CVE and CVSS.

Governance moved from MITRE to the Center for Internet Security and, after a period of dormancy, to a community organisation ([OVAL-Community/OVAL](#) on GitHub). Two recent releases matter:

- **OVAL 5.12** (4 Dec 2024) — the first update in several years; incremental schema additions.
- **OVAL 6.0** (24 Jan 2025) — the first "major" release in ~20 years. It *removes* backward compatibility with 5.x content, drops ~140 deprecated tests, and introduces the **encapsulated definition** (bundling a definition with the tests/objects/states it needs, for portability and easier maintenance).

The tooling gap

OpenSCAP — the reference open-source engine that Red Hat, Ubuntu, SUSE, Oracle Linux and most compliance pipelines rely on — still advertises `OVAL_SUPPORTED 5.11.3` (as of 1.4.3). Support for 5.12/6.0 is discussed as part of a future SCAP 3.0 cycle but not shipped. So the language has just made a *breaking* jump that its primary open evaluator cannot yet consume. Content authors must therefore target 5.11 in practice, regardless of what the latest spec says.

The encapsulated-definition idea in 6.0 is, in fact, a tacit admission of OVAL's core ergonomic problem: content is fragmented across separately-referenced objects/states/variables, so a single logical check is hard to move, review, or reason about. OVAL-NG takes that insight further.

3 OVAL in practice — the XORCISM experience

XORCISM is a cyber-fusion / CTEM platform with an endpoint agent. It both *imports* OVAL content and *evaluates* it on hosts, so it has felt OVAL's strengths and its costs directly. Three observations are especially instructive because they were discovered the hard way, in code.

3.1 Metadata import is not the same as an evaluable model

An importer can ingest the OVAL Definitions stream into a database (~35,000 definitions, ~140,000 tests, ~66,000 CVE links) and still be **unable to evaluate any of it**. In XORCISM the objects, states, object/test linkage and entity rows came out empty — only definition metadata and a criteria skeleton survived round-tripping through a relational model. The lesson: OVAL's value lives in the *object↔state↔test* wiring, and that wiring is precisely the part that is most awkward to represent, transport and persist. A format whose "easy" parts are the worthless parts is a format that fights its implementers.

3.2 One mature evaluator, so everyone re-implements the algebra

Because OpenSCAP's Windows OVAL probe coverage is partial, XORCISM had to write a **native OVAL 5.x evaluator from scratch** for Windows endpoints — implementing the full result algebra (AND / OR / ONE / XOR truth tables, criterion negation, `extend_definition` with cycle-guarding, the existence/state check enumerations, the state operations across `int/string/boolean/version` datatypes, and the variable

system). That this was necessary at all — and that it is a recurring story across vendors — is the strongest possible signal that the *specification surface is too large for the ecosystem it serves*. A good standard should make the second, third and fourth implementations cheap.

3.3 The real content comes from distros, not from "OVAL"

In production, evaluable content is pulled from **distribution feeds** — Ubuntu's OVAL, Debian's, the RHEL/ Oracle Linux streams — not from a central OVAL repository. Each ships large, compressed, platform-specific XML. The agent fetches, decompresses, and hands them to `oscap`; XORCISM additionally caches and serves this content to offline agents. The practical centre of gravity is "per-platform vulnerability feeds in OVAL XML", which argues for a format whose **platform modules are first-class, independently versioned units** rather than namespaces bolted onto one schema.

What XORCISM proves about OVAL

The semantics are worth keeping (XORCISM re-implemented them faithfully and they work); the **encoding, the schema sprawl, and the single-evaluator reality** are the problems. OVAL-NG is designed around exactly these three findings.

4 The open-source scanner landscape

TOOL	STATUS	OVAL SUPPORT & NOTES
OpenSCAP (<code>oscap</code>)	Active, dominant	The reference FOSS engine. Mature Linux probes; OVAL 5.11.3; partial Windows probe coverage. SCAP datastream / XCCDF / ARF. Effectively the <i>de facto</i> definition of "what OVAL means" in practice.
jOVAL / Joval	Dormant / commercial	Once the cross-platform alternative (strong Windows/network device coverage). Went commercial, then largely inactive — a cautionary tale about sustaining a full OVAL implementation outside a vendor.
CIS-CAT Pro	Proprietary	Consumes OVAL+XCCDF for CIS Benchmarks; closed source, licensed. Not a community evaluator.
Wazuh	Active	Uses OVAL/distro CVE feeds for its vulnerability detector but with its <i>own</i> matching engine — i.e. it consumes OVAL <i>data</i> without implementing OVAL <i>evaluation</i> . Another sign the full model is too heavy to adopt wholesale.
XORCISM agent	Active (this platform)	Delegates to <code>oscap</code> where present; ships a purpose-built native OVAL evaluator for Windows hosts without OpenSCAP. Proof that even a focused team must re-implement the algebra to get cross-platform coverage.

The landscape has **one** healthy general-purpose open evaluator. Everything else either consumes OVAL as data, re-implements a subset, or is proprietary. A modernization that lowers the cost of a *conformant* implementation is therefore the single highest-leverage change available to the ecosystem.

5 Problem analysis — ten friction points

#	FRICTION	WHY IT HURTS IMPLEMENTERS / AUTHORS / MAINTAINERS
1	XML-Schema sprawl	Dozens of <code>.xsd</code> files (core + independent + windows + unix + linux + macos + ...). Toolchains need a full XSD stack; partial support is the norm.
2	Indirection overload	A single check fans out across <code>definition</code> → <code>test</code> → <code>object</code> → <code>state</code> → <code>variable</code> by id reference. Reviewing one logical assertion means chasing five elements across a document.
3	Per-platform schema coupling	Adding a new platform or probe means editing the central schema family and cutting a new language version — a heavyweight, serialized process.
4	Verbose, namespace-heavy serialization	XML namespaces, <code>xsi:type</code> discriminators and boilerplate dwarf the actual logic; hand-authoring is error-prone without a schema-aware IDE.
5	Result algebra is under-documented for implementers	The AND/OR/ONE/XOR tables, <code>check_existence</code> vs <code>check</code> , and "not evaluated/not applicable/error" propagation are correct but scattered; everyone re-derives them.
6	Variables are a second language	Constant/local/external variables with component functions form a mini expression language expressed in XML — powerful, but a large extra surface to implement.
7	Weak machine-readable security linkage	CVE/CPE/CVSS live in free-text metadata and <code><reference></code> elements, not as typed, validated fields — so consumers parse heuristically.
8	No conformance tiers	"OVAL support" is all-or-nothing in spirit, so tools advertise vague, partial compliance and content authors cannot rely on a guaranteed baseline.
9	Spec/tooling drift	6.0 broke 5.x while the main evaluator stayed at 5.11 — authors cannot safely target the latest spec.
10	Testing & CI are hard	There is no lightweight, language-level fixture format ("given this fake system, the definition must yield <i>true</i> "), so probe correctness is validated ad hoc.

6 Design goals

1

Keep the semantics

2

Shrink the surface

3

Modular platforms

4

Lossless bridge

1. **Preserve OVAL's evaluation semantics** — the typed object/state model and the result algebra are an asset; OVAL-NG re-expresses them, it does not reinvent them.
2. **JSON-native, schema-validated** — one small set of JSON Schema files; any modern stack validates and round-trips it without bespoke XSD tooling.
3. **Locality over indirection** — a check is readable in one place; references remain possible for reuse but are never *required* to understand a single assertion (generalising OVAL 6.0's "encapsulated definition").
4. **Platforms as versioned modules** — probe behaviour lives in independently-versioned *probe packs* discovered through a capability registry, so adding macOS 15 or a cloud API probe never forces a core release.

- 5. **Security data is typed** — CVE, CPE 2.3, CVSS vectors, EPSS, CWE and KEV are first-class, validated fields, not free text.
- 6. **Tiered conformance** — a tiny mandatory Core plus optional profiles, so a tool can claim *exactly* what it supports and content can require a guaranteed baseline.
- 7. **Backward bridge, not a fork** — a deterministic translator turns existing OVAL 5.x/6.0 XML into OVAL-NG, preserving evaluation outcomes, so the enormous body of distro content is reused on day one.
- 8. **Testable by construction** — a built-in fixture format lets a definition ship with "mock system → expected result" cases that any evaluator runs as a conformance/CI check.

7 Proposal: OVAL-NG

7.1 Serialization & the document envelope

OVAL-NG documents are JSON (YAML permitted as an authoring convenience; canonical form is JSON). A document declares the language version, the *profiles* it uses, and the *probe packs* it depends on — making capability requirements explicit and machine-checkable before evaluation starts.

```
// vulnerability/CVE-2024-3400.oval.json
{
  "oval_ng": "1.0",
  "kind": "definition-bundle",
  "profiles": ["core", "vuln"],
  "requires_packs": [{ "id": "linux.dpkg", "min": "1.0" }],
  "generator": { "product": "XORCISM", "timestamp": "2026-06-21T10:00:00Z" },
  "definitions": [ /* ... self-contained, see §7.3 ... */ ]
}
```

7.2 The object model — five kinds, not fifty schemas

Everything is one of five JSON object kinds. Platform specifics are pushed *out* of the core grammar and *into* the typed payload of a `probe` and an `expectation`, which the relevant probe pack defines and validates.

KIND	ROLE (OVAL 5.X ANALOGUE)
<code>definition</code>	A named assertion with metadata + a <code>criteria</code> tree (OVAL <i>definition</i>).
<code>check</code>	A leaf assertion: a <code>probe</code> (what to collect) + an <code>expect</code> predicate (the comparison). Fuses OVAL <i>test</i> + <i>object</i> + <i>state</i> into one local, readable unit.
<code>probe</code>	A typed "collect" request handled by a probe pack — e.g. <code>{"pack": "linux.dpkg", "query": {"name": "openssl"}}</code> (OVAL <i>object</i>).
<code>var</code>	An optional named value or expression for reuse (OVAL <i>variable</i>), now using the §7.4 expression grammar.
<code>fixture</code>	NEW A mock system + expected verdict for a definition — language-level tests (no OVAL analogue).

Note what is gone: there is no separate *test*, *object*, *state* and *system_characteristics* schema family. Collection results are produced by probe packs and matched by the `expect` predicate inline. Reuse via `$ref` remains available, but it is opt-in, not the default.

7.3 Criteria & the `expect` predicate

Criteria are an explicit tree of boolean nodes. The result algebra (AND/OR/ONE/XOR, negation, the existence/state check enumerations, and the propagation of `true` / `false` / `unknown` / `not_applicable` / `error`) is normative and specified once, in pseudocode, in the Core profile.

```
// a self-contained definition (no cross-document chasing)
{
  "id": "def:cve-2024-3400",
  "class": "vulnerability",
  "title": "Command injection in PAN-OS GlobalProtect (CVE-2024-3400)",
  "metadata": {
    "cve": ["CVE-2024-3400"],
    "cpe": ["cpe:2.3:o:paloaltonetworks:pan-os:*:*:*:*:*:*"],
    "cvss_v31": "CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H",
    "cwe": ["CWE-77"], "kev": true, "epss": 0.94
  },
  "criteria": {
    "op": "AND",
    "of": [
      { "check": {
        "probe": { "pack": "linux.os-release", "query": {} },
        "expect": { "field": "id", "op": "equals", "value": "pan-os" } } },
      { "check": {
        "probe": { "pack": "linux.dpkg", "query": { "name": "globalprotect" } },
        "expect": { "field": "version", "op": "version_lt", "value": "6.1.1" },
        "exists": "at_least_one" } }
    ]
  }
}
```

One screen, one assertion. The CVE/CPE/CVSS/EPSS/KEV linkage is typed; the platform detail lives in the probe packs (`linux.os-release` , `linux.dpkg`), not in the grammar.

7.4 Expressions for variables & values

OVAL's variable "component functions" become a small, documented expression grammar reused everywhere a value is needed (variables, `expect.value` , probe query fields). It supports literals, references (`$var` , `probe.field`), and a fixed function set (`concat` , `substr` , `regex_capture` , `split` , `count` , `arithmetic` , `glob_to_regex`). Crucially it is a *closed* grammar — no Turing-completeness, so it stays auditable and safe to evaluate.

```
{ "id": "var:min_ver", "expr": { "fn": "concat", "args": ["6.", "$branch", ".1"] } }
```

7.5 Probe packs & the capability registry

A **probe pack** is a versioned module declaring (a) the probe kinds it provides, (b) the JSON Schema for each probe's `query` and the fields it returns, and (c) a platform applicability predicate. Packs are identified by reverse-DNS-style ids (`linux.dpkg` , `windows.registry` , `macos.plist` , `cloud.aws.ec2` , `net.snmp`). Evaluators advertise the packs and versions they implement; a document's `requires_packs` is matched against that set *before* evaluation, turning today's silent "not evaluated" into an explicit, up-front capability check.

Why this is the keystone change

Decoupling platform coverage from the core grammar means a new probe (a cloud API, a container image, a new OS release) ships as a *pack* on its own cadence — no central schema edit, no new language version, no flag day. It directly fixes friction points 1, 3 and 9, and it gives small evaluators a legitimate way to support "Core + the three packs we care about" and say so precisely.

7.6 Conformance profiles

PROFILE	GUARANTEES
core (mandatory)	Document envelope, the five kinds, the criteria algebra, the expression grammar, the results model, and fixtures. No probes.
vuln	Typed security metadata + the matching helpers (version comparison, CPE applicability) needed for CVE checks.
config	Compliance semantics (pass/fail rationale, remediation hints, benchmark cross-refs for XCCDF/CIS).
inventory	Asset/CPE emission semantics for software & platform discovery.

A tool declares, e.g., `core + vuln + {linux.dpkg, linux.rpm, windows.registry}` — a precise, testable claim. Content declares what it needs; mismatches fail loudly at load time.

7.7 Results model

Results are JSON mirroring the definition tree, with each node carrying its verdict and the collected evidence that produced it — so a result is self-explaining and diff-able, and maps cleanly onto storage such as XORCISM's `OVALRESULTS` (per-asset verdict, class, CVE/CPE fan-out).

```
{ "kind": "results", "asset": "fw-01", "definition": "def:cve-2024-3400",
  "verdict": "true", "class": "vulnerability",
  "evidence": [ { "probe": "linux.dpkg", "collected": { "name": "globalprotect", "version": "6.1.0" },
                  "expect": "version_lt 6.1.1", "verdict": "true" } ],
  "emit": { "cve": ["CVE-2024-3400"] } }
```

7.8 The backward-compatibility bridge

A normative, deterministic translator (`oval5→ng`) maps OVAL 5.x/6.0 XML to OVAL-NG: each `test+object+state` triple becomes a `check`; platform schemas map to probe packs (the mapping table is part of each pack); variables become expressions; criteria map one-to-one. Because the algebra is preserved, **evaluation outcomes are identical**, which is itself a conformance test: translate the public distro feeds, evaluate both forms, assert equal verdicts. This lets an OVAL-NG engine consume the entire existing content corpus (Ubuntu/Debian/RHEL/SUSE/Oracle feeds) from day one while new content is authored natively.

7.9 Reference-implementation footprint

The Core profile is deliberately small enough that a **conformant evaluator core fits in a few hundred lines** in any modern language: parse+validate (delegated to a JSON Schema library), walk the criteria tree, dispatch each `probe` to a registered pack, apply the `expect` predicate, fold verdicts through the algebra, emit results. Probe packs are independent plug-ins. XORCISM's existing native Windows evaluator already

implements the hard part (the algebra + registry/file/version probes); under OVAL-NG that work becomes the `windows.registry` / `windows.file` packs plus the shared Core — i.e. the platform's prior investment is *reused*, not thrown away.

8 Worked examples

8.1 A Windows configuration check (compliance)

```
{
  "id": "def:win-smbv1-disabled",
  "class": "compliance",
  "title": "SMBv1 server is disabled",
  "metadata": { "benchmark": ["CIS Microsoft Windows: 18.3.1"] },
  "criteria": { "check": {
    "probe": { "pack": "windows.registry",
      "query": { "hive": "HKLM",
        "key": "SYSTEM\\CurrentControlSet\\Services\\LanmanServer\\Parameters",
        "name": "SMB1" } },
    "expect": { "field": "value", "op": "equals", "value": 0, "datatype": "int" },
    "exists": "at_least_one",
    "on_absent": "false"          // missing value ⇒ not compliant
  } }
}
```

8.2 A definition shipping its own tests (fixtures)

```
{
  "for": "def:win-smbv1-disabled",
  "kind": "fixture",
  "cases": [
    { "name": "disabled-explicitly",
      "system": { "windows.registry": [ { "key": "...\\Parameters", "name": "SMB1", "value": 0 } ] },
      "expect_verdict": "true" },
    { "name": "enabled",
      "system": { "windows.registry": [ { "key": "...\\Parameters", "name": "SMB1", "value": 1 } ] },
      "expect_verdict": "false" },
    { "name": "value-absent",
      "system": { "windows.registry": [] },
      "expect_verdict": "false" }
  ]
}
```

Any conformant evaluator runs these in-memory: no live host required, so probe and content correctness become ordinary unit tests in CI — directly fixing friction point 10.

8.3 Declaring a probe pack

```
{
  "kind": "probe-pack", "id": "linux.dpkg", "version": "1.0",
  "applies_when": { "pack": "linux.os-release", "expect": { "field": "family", "op": "equals",
"value": "debian" } },
  "probes": {
    "": { "query_schema": { "name": { "type": "string", "required": true } },
      "returns": { "name": "string", "version": "evr", "arch": "string", "installed":
"boolean" } }
  }
}
```

9 Why this is easier to implement & maintain

CONCERN	OVAL 5.11 / 6.0	OVAL-NG
Validation tooling	Full multi-file XSD stack; namespace-aware parsers	Stock JSON Schema validator in any language
Reading one check	Chase def→test→object→state→variable by id	One self-contained <code>check</code> object
Adding a platform/probe	Edit central schema family; cut a language version	Publish a versioned probe pack independently
Advertising support	Vague "partial OVAL support"	Precise: <code>core+vuIn+{packs}</code>
Minimal evaluator	Large; few have ever completed one	Core ≈ a few hundred lines + plug-in packs
Testing content	Ad hoc, needs real/golden systems	Built-in fixtures run in CI
Security data	Free-text references, heuristic parsing	Typed CVE/CPE/CVSS/EPSS/CWE/KEV
Existing content	6.0 breaks 5.x	Lossless bridge reuses all 5.x/6.0 feeds

10 Migration & adoption path

1. **Phase 0 — Core spec + schemas.** Publish the Core profile, JSON Schemas, the normative algebra pseudocode, and a conformance test-suite (a corpus of fixtures). Stand up a reference Core evaluator.
2. **Phase 1 — Bridge.** Ship `oval5→ng` and the pack mapping tables for the common platforms (dpkg, rpm, windows.registry/file, macos.plist, independent/textfilecontent, family, environmentvariable). Validate by equal-verdict testing against public distro feeds.
3. **Phase 2 — Probe packs.** Community-maintained packs per platform on independent cadences; a public registry of pack ids/versions and the evaluators that implement them.
4. **Phase 3 — Native authoring.** Distro and benchmark authors emit OVAL-NG directly; the bridge remains for legacy. SCAP 3.0 can reference OVAL-NG profiles instead of an OVAL version number.

XORCISM as an early adopter / reference consumer

XORCISM can adopt OVAL-NG incrementally with low risk: (1) its agent's native evaluator is refactored into a Core engine + `windows.*` packs, reusing existing algebra/probe code; (2) its content service runs the bridge so cached distro feeds are exposed in both XML and OVAL-NG; (3) the `OVALRESULTS` store already matches the results model (per-asset verdict, class, CVE/CPE fan-out), so ingestion is a thin mapping; (4) the `openscap` connector keeps consuming legacy ARF in parallel. This makes XORCISM a working reference for "consume legacy via bridge, author new natively".

11 Governance & versioning

- **SemVer the Core, independently version packs.** The Core grammar/algebra changes rarely and carefully; platform reality changes constantly and should not be gated on Core releases.
- **Deprecate by capability, not by flag day.** Retiring a probe means a pack major bump that evaluators opt into — avoiding the 5.x→6.0 "break everything at once" problem.
- **Conformance is earned by passing the suite,** not asserted in a README. The fixture corpus *is* the definition of conformance.
- **Keep a permanent bridge.** The 20+ year OVAL corpus is an asset; the translator is a first-class, maintained deliverable, not a one-off.

12 Risks & open questions

- **Ecosystem inertia.** OpenSCAP is the gravity well. Adoption hinges on a bridge good enough that OVAL-NG is "free" to try and on at least one credible reference evaluator — ideally with OpenSCAP able to emit OVAL-NG results.
- **Probe-pack fragmentation.** Independent packs risk divergence; mitigated by a central registry, mandatory query/return schemas, and shared fixtures per pack.
- **Expression grammar scope creep.** Must stay closed and non-Turing-complete to remain safe and cheap to implement; resist requests for general scripting.
- **EVR / version semantics.** Cross-distro version comparison (epoch:version-release, Debian vs RPM) is genuinely hard and must be specified precisely in the `vuln` profile, with fixtures.
- **SCAP alignment.** OVAL-NG should slot into SCAP 3.0 cleanly alongside XCCDF/CPE rather than compete with it; coordination with that effort is a prerequisite, not an afterthought.

Appendix A OVAL 5.x → OVAL-NG construct map

OVAL 5.X / 6.0	OVAL-NG
<code>oval_definitions</code> document	<code>definition-bundle</code> document
<code>definition</code> (+ class)	<code>definition</code> (kind), <code>class</code> field
<code>criteria</code> / <code>criterion</code> / <code>extend_definition</code>	<code>criteria</code> tree (<code>op/of</code> , <code>check</code> , <code>\$ref</code>)
<code>test</code> + <code>object</code> + <code>state</code>	one <code>check</code> = <code>probe</code> + <code>expect</code> (+ <code>exists</code>)
platform schemas (windows/unix/linux/...)	probe packs (<code>windows.*</code> , <code>linux.*</code> , ...)
<code>variable</code> + component functions	<code>var</code> + closed expression grammar
<code>system_characteristics</code>	probe-pack collected fields (transient, in results)
<code>oval_results</code> + directives	<code>results</code> document (self-explaining tree)
<code>check</code> , <code>check_existence</code> , <code>entity_check</code>	<code>expect</code> + <code>exists</code> + <code>on_absent</code>
OVAL 6.0 <i>encapsulated definition</i>	the default: every <code>definition</code> is self-contained
<code><reference source="CVE"></code>	typed <code>metadata.cve[]</code> (+ <code>cpe/cvss/cwe/epss/kev</code>)

Discussion draft v0.9 · 21 June 2026. Prepared to inform the OVAL Community and SCAP 3.0 discussions; not an official OVAL deliverable. "OVAL" is a trademark of its respective stewards; references to OVAL 5.x/6.0, OpenSCAP, CIS-CAT, jOVAL and Wazuh are for analysis only.